Inhalt

- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW

Kapitel 7

Transportschicht

7.1 Einführung Transportschicht (Layer 4)

Inhalt

- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW

Aufgaben

- Ermöglicht die *Kommunikation zwischen Anwendungen*, die auf verschiedenen oder gleichem Hosts laufen
  - Adressierung individueller Anwendungen auf einem Host (Multiplexing)
    - Auf einem Host können mehrere Anwendungen laufen
  - Anwendungen erscheint es, als ob sie direkt miteinander verbunden wären („logischer“ Ende-zu-Ende-Transport)
    - Router, Switches etc. für Anwendung nicht sichtbar („transparent“)
  - Zurverfügungstellung einer **Programmierschnittstelle** (**API**, Application Programming Interface), über die Anwendungen Daten austauschen können

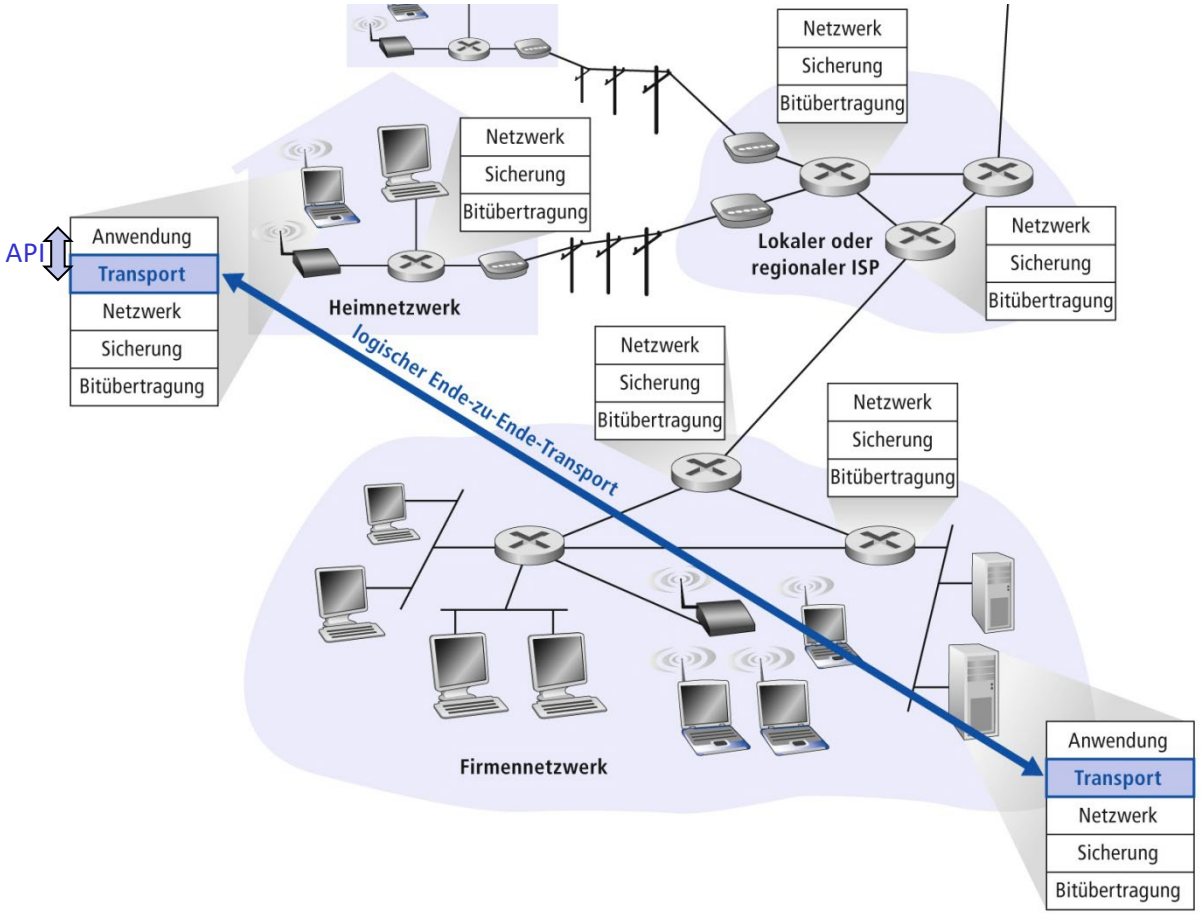


Abbildung 3.1: Die Transportschicht realisiert logische, nicht physische Kommunikation zwischen Anwendungsprozessen.

Aus Computernetzwerke von Kurose und Ross

Inhalt

- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW

Aufgaben

- Zurverfügungstellung verschiedener Dienstqualitäten des logischen Ende-zu-Ende-Datentransports
  - Best effort: Fire and Forget ala IP, vs.
  - Zuverlässiger Datendienst
    - Daten kommen garantiert bei der Peer-Anwendung unverfälscht an
    - In der richtigen Reihenfolge
    - Genau einmal (keine Duplikate)
  - Flusskontrolle
    - Schneller Sender wird auf Geschwindigkeit eines langsamen Empfängers gebremst
  - Überlastkontrolle
    - Netzwerkelemente (Router, Switche etc) werden vor Überlast geschützt
  - Datensicherheit
    - Verschlüsselung
    - Signierung
    - Authentifizierung
    - …
  - ….

7.1 Einführung Transportschicht (Layer 4)

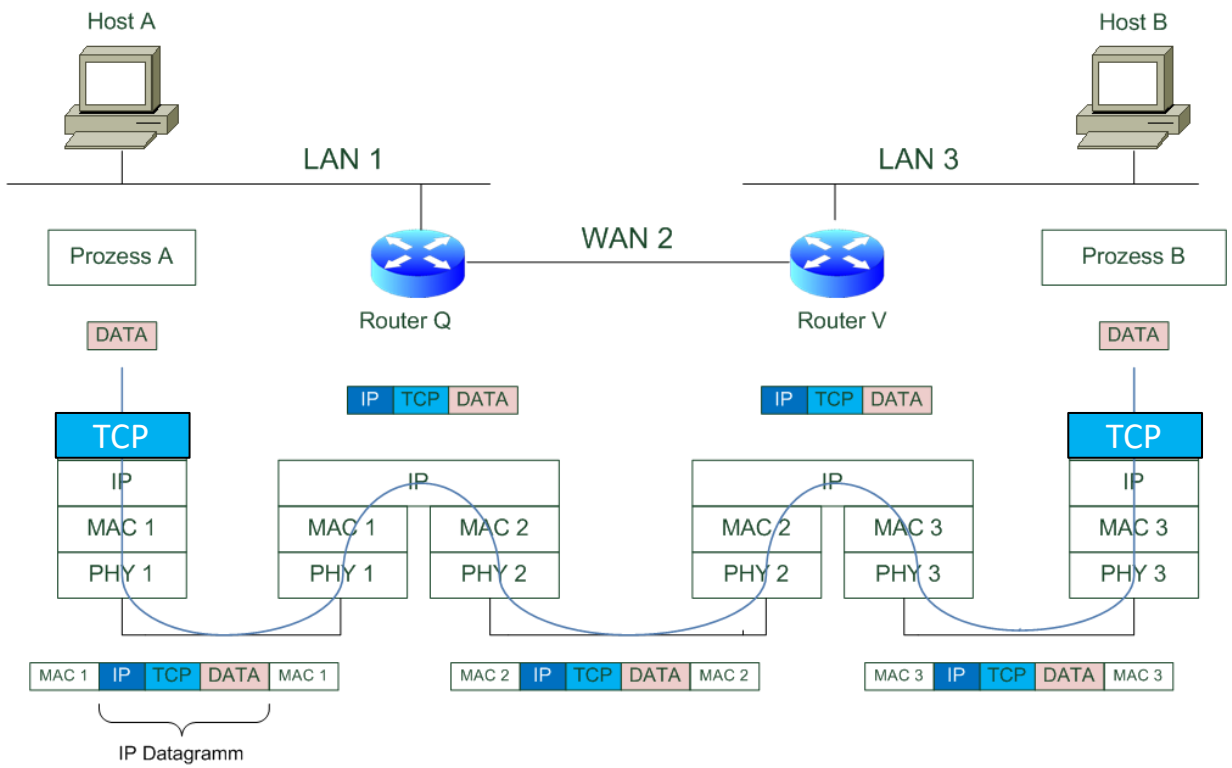
Inhalt

- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW

Warum eigene Schicht?

- Logik lässt sich auf Endsystemen alleine (= Empfänger und Sender) realisieren
- Netzwerkgeräte nicht betroffen
- Layer 3 (IP) betrifft aber auch Netzwerkgeräte

→ Realisierung als eigener Schicht, die die Dienste der IP-Schicht verwendet



Aus Kapitel 6.3: Die Transportschicht (hier als TCP-Realisierung) ist nur auf den Endgeräten A und B aktiv, nicht auf den Routern Q und V bzw. Switchen (nicht gezeigt)

7.2 Adressierung von Anwendungen: Multiplexing

Inhalt

- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW

- IP kann mehrere Anwendungen auf dem gleichen Host nicht unterscheiden
  - Warum? Die Quell- und Ziel-IP-Adresse identifiziert nur einen Host-Rechner (genauer: Netzwerkkarte)
- Anwendungen sind betriebssystemabhängig implementiert
  - Prozesse, Prozess-Ids, Threads, Speicherzugriff… sind Betriebssystemkonzepte
- Transportschicht soll aber betriebssystemunabhängig sein
  - Linux Anwendung auf Host A soll Daten mit Windows Anwendung auf Host B austauschen können
  - Anwendungen sollen das Betriebssystem des Kommunikationspartners gar nicht kennen müssen
- Internet führt Konzept der "**Sockets**" ein
  - Betriebssystemunabhängige Andockstellen an die Transportschicht für Anwendungen, die über ein internet kommunizieren wollen
  - Pro Host (und pro Transportprotokoll) werden 2^16 =65536 Sockets angeboten

    →Pro Host können bis zu 65536 Anwendungen (pro Transportprotokoll) unterstützt werden
  - Sockets werden durchnummeriert von 0 bis 65535 (***Socket-ID*** bzw***. Port-ID***)
  - Die Port-ID des Senders und Empfängers werden im Transport-Header übertragen
  - API zur Datenübertragung nennt sich „**Socket API**" bzw „**Socket Programmierung**"

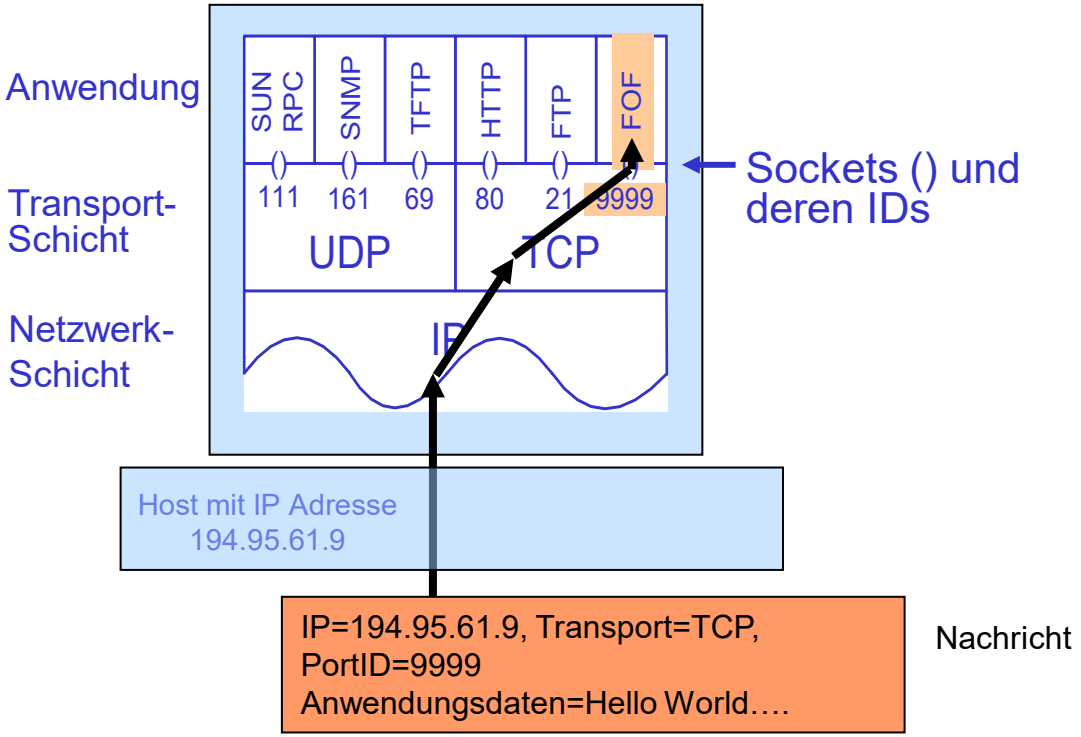
7.2 Adressierung von Anwendungen: Multiplexing

Inhalt

- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW

Multiplexing:

- Empfänger:
  1. Anwendung registriert sich bei der Transportschicht mittels Socket-API, z.B. „`new TCP.Socket(PortID=9999)`“ (Details siehe Socket-Programmierung)
  2. Erhält die Transportschicht eine Nachricht von der Netzwerkschicht, stellt sie die Nachricht der Anwendung zur Verfügung, die sich unter der PortID aus dem Transport Header registriert hat.
- Sender:
  1. Anwendung registriert sich bei der Transportschicht (wie Empfänger, aber meist mit andere PortID)
  2. Anwendung übergibt Anwendungsdaten an Transportschicht unter Angabe der Empfänger PortID. Die Transportschicht fügt diese in ihren Header ein.



- Bemerkungen:
  - ***Socket:*** PortID zusammen mit der IP-Adresse identifiziert eine Anwendung weltweit.
  - Übliche Darstellung eines Sockets IP:PortID
    - IPv4: 194.95.61.9:9999
    - IPv6: [2001:1::1]:9999

7.3 Gebräuchliche Transportprotokolle und deren Eigenschaften

Inhalt

- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW

- Im Internet gibt es eine Vielzahl von Transportprotokollen
  - UDP (User Datagram Protocol)
  - TCP (Transport Control Protocol)
  - TLS (Transport Layer Security)
  - RTP (Realtime Transport Protocol)
  - MPTCP (Multipath TCP)
  - SCTP (Stream Control Transmission Protocol)
  - DCCP (Datagram Congestion Control Protocol)
  - QUIC (Quick UDP Internet Connections)
  - …
- Alle haben verschiedene Eigenschaften

7.3 Gebräuchliche Transportprotokolle und deren Eigenschaften

Inhalt

- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW

Eigenschaft	UDP	TCP	TLS	RTP	MPTCP	SCTP	DCCP	QUIC
Verbindungs-orientiert	Nein	Ja	Ja	Nein	Ja	Ja	Nein	Ja
Nachrichten-/Stream-orientiertes API	Nachrichten	Stream	Stream	Nachrichten	Stream	Nachrichten	Nachrichten	Stream
Dienstqualität	Best effort	Zuverlässig	Zuverlässig	Best effort	Zuverlässig	Zuverlässig	Best effort	Zuverlässig
Echtzeitfähig	Bedingt	Nein	Nein	Ja (keine harte)	Nein	Nein	Bedingt	Nein
Multicast-unterstützung	Ja	Nein	Nein	Ja	Nein	Nein	Nein	Nein
Flusskontrolle	Nein	Ja	Ja	Nein	Ja	Ja	Nein	Ja
Überlast-kontrolle	Nein	Ja	Ja	Nein	Ja	Ja	Nein	Ja
IT-Sicherheit	Keine	Wenig	Ja	Keine	Wenig	Wenig	Keine	Ja

➢ Die gebräuchlichsten Transportprotokolle sind UDP und TCP

7.4 Das User Datagram Protocol (UDP)

Inhalt

- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW

- UDP ist/hat:
- End-to-end
  - UDP-Logik nur im Sender und Empfänger aktiv
- Verbindungslos (siehe 7.5)
- Nachrichtenorientiert
  - Eine UDP Anwendung sendet und empfängt individuelle Nachrichten
- Best-effort Dienstqualität
  - fast wie IP
    - Nachrichten können ohne Rückmeldung verloren bzw. dupliziert werden
    - Reihenfolge der Nachrichten ist nicht garantiert
    - Keine Flusskontrolle, keine Überlastkontrolle (siehe Kapitel TCP)
    - Aber: UDP kann Fehler in den Anwendungsdaten erkennen
- Multicast support
  - Anwendung kann Nachricht an Gruppe von Empfängern senden (Ziel-IP = Multicast-Adresse)
  - Anwendung kann Nachrichten von beliebigen Sendern erhalten
- Keine IT-Sicherheit
  - Nachrichten werden nicht verschlüsselt, authentifiziert oder autorisiert

7.5 Verbindungsloses Paradigma

Inhalt

- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW

- Eine UDP Anwendung braucht keine Verbindung aufbauen bevor sie Daten senden kann
- UDP erlaubt der Anwendung beliebig lange Pausen zwischen zwei Übertragungen
- Falls Anwendung keine Daten sendet
  - sendet UDP auch keine
- UDP verwendet keine Kontrollnachrichten
- UDP hat extrem kleinen Overhead
- UDP verzögert keine Nachrichten (z.B. weil es auf eine vorherige wartet)
  - Bedingt Echtzeitfähig

7.5 Nachrichtenorientiertes Interface

Inhalt

- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW

- UDP bietet der Anwendung ein message-oriented interface (MOI)
- UDP zerlegt/verbindet keine Anwendungsnachrichten
  - Eine Anwendungsnachricht bedingt genau eine UDP Nachricht und umgekehrt
- Das MOI hat einige wichtige Konsequenzen:
  - UDP Anwendungen erkennt Datengrenze zwischen zwei Anwendungsnachrichten
  - Jede UDP Nachricht muss in ein IP Datagram passen
  - IP Datagramgröße bedingt maximale Größe einer Anwendungsnachricht
- Terminologie: UDP Nachrichten/Pakete nennt man auch (UDP-) Datagramme
- Ungünstige Nachrichtengröße führt zu schlechter Ausnutzung des Netzwerks
  - Zu kleine Nachrichten
    - Datagramme haben ungünstiges Header/Daten Verhältnis
  - Zu große Nachrichten
    - Datagramme sind größer als die Netzwerk MTU und warden fragmentiert werden

7.6 UDP Verwendung

Inhalt

- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW

- UDP korrigiert keine Übertragungsfehler
  - Eine UDP-Anwendung muss immune dagegen sein
  - oder eigene Fehlererkennung und –korrektur durchführen
- UDP verzögert keine Nachrichten, keine Zwischenspeicherung
- UDP ist geeignet für Anwendungen wie Live Sprach- oder Videoübertragung
  - können Übertragungsverluste verkraften
  - aber keine Verzögerungen

7.7 UDP Datagramm Format

Inhalt

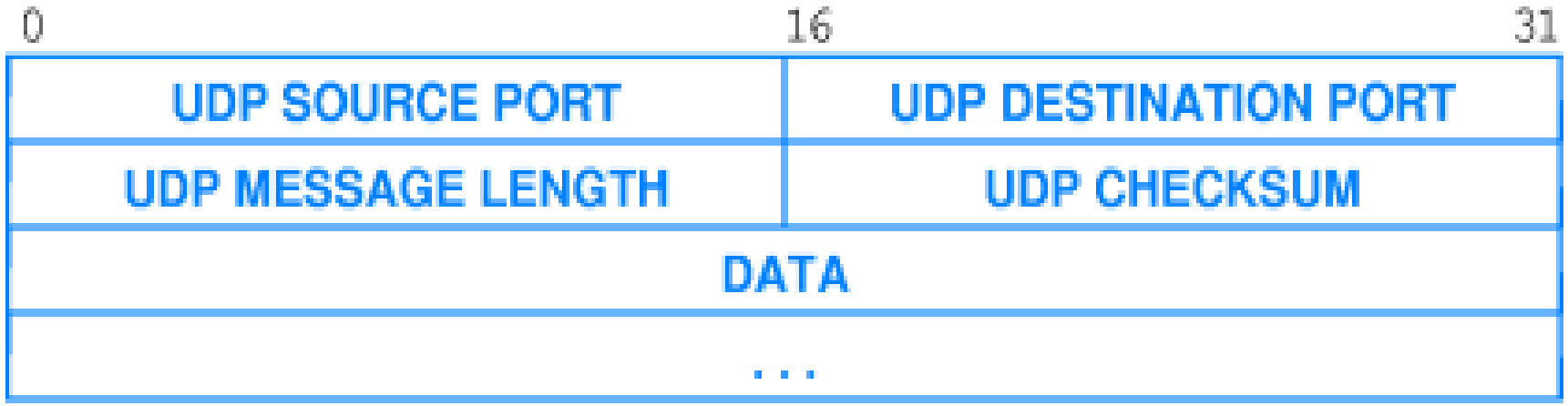
- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW

- UDP Nachricht nennt man auch UDP-Datagramm und hat zwei Teile:
  - Einen kleinen Header, der den Sender/Empfänger spezifiziert
  - Nutzdaten der Anwendung
- Figure 24.1 zeigt das UDP-Datagram Format.
  - SOURCE PORT (Quell-SocketID)
  - DESTINATION PORT (Ziel-SocketID)
  - MESSAGE LENGTH (Datagramlänge)
  - CHECKSUM
  - PAYLOAD (Nutzdaten)

7.7 UDP Datagram Format

Inhalt

- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW



Picture taken from Comer

Figure 24.1 The format of a UDP user datagram. Each user datagram begins with an eight octet header followed by the data being sent.

7.8 UDP Checksumme und der Pseudo IP Header

Inhalt

- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW

- CHECKSUM-Feld is optional
  - Sender kann Checksumme berechnen
  - oder jedes Bit der Checksumme auf Null setzen
- Das Ziel untersucht die Checksumme
  - Überprüft die Checksumme falls ihr Wert ungleich Null ist
- UDP berechnet die Checksumme über
  - Nutzdaten
  - UDP-Header (Ports und Länge)
  - Pseudo IP-Header
    - Quell-IP
    - Ziel-IP
    - Und Typ-Felder

7.9 UDP Encapsulation

Inhalt

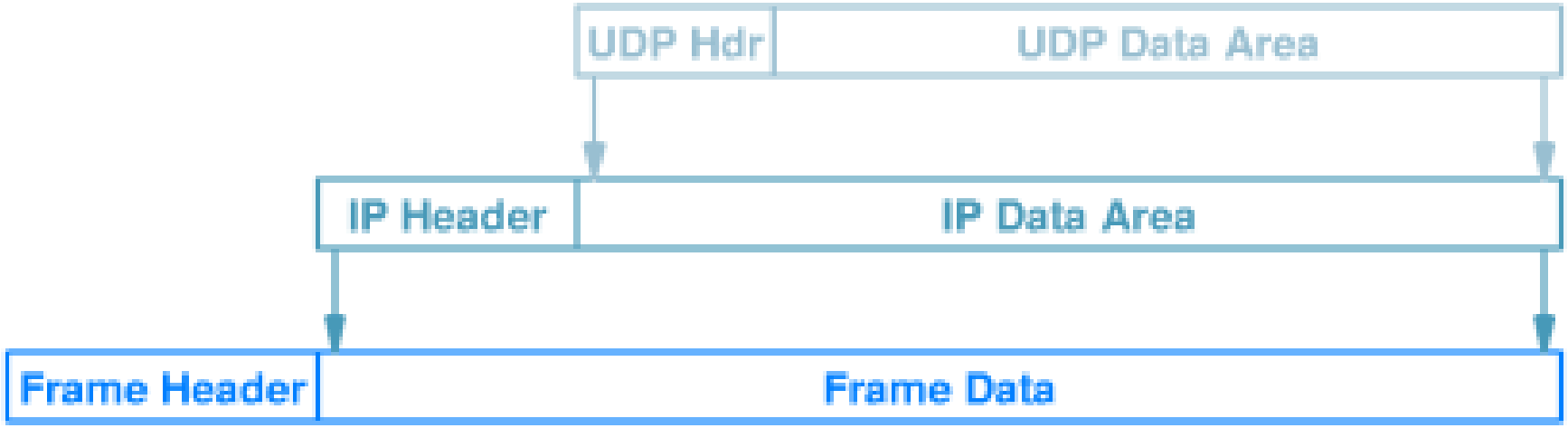
- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW

- UDP verwendet IP für seine Übermittlungsaufgaben
- Wie ICMP, verwendet UDP die Technik des “Encapsulations” (Einpackens)
  - UDP Datagramm in ein IP Datagramm und schlussendlich in ein LAN-Frame
- Figure 24.2 zeigt dieses Konzept

7.9 UDP Encapsulation

Inhalt

- Grundlagen
- Pakete, Rahmen, Fehlererkennung
- LAN-Technologien
- Routing
- IP-Adressen
- IP
- **Transportschicht/UDP**
- TCP
- DNS
- DHCP
- WWW



Picture taken from Comer

Figure 24.2 The encapsulation of a UDP message in an IP datagram. The entire UDP message, including the header and data areas resides in the data area of the IP datagram.